

Lab Manual: Building a Flask Webhook Receiver with SQLite

Objective

By the end of this lab, you will:

- Understand how webhooks work.
- Create a Flask app with **dedicated webhook endpoints**.
- Store incoming webhook data into an SQLite database.
- Test webhook ingestion using `curl`.

Prerequisites

- Python 3.7+ installed.
- Flask installed: `pip install flask`

Step 1: Set Up Project Folder

```
mkdir webhook_demo
cd webhook_demo
```

Inside, create these files:

```
webhook_demo/
|— app.py
|— database.py
```

Step 2: Create the Database Module

Create `database.py`:

```
import sqlite3
```

```

# Initialize database and create table
def init_db():
    conn = sqlite3.connect("webhooks.db")
    cursor = conn.cursor()

    cursor.execute("""
        CREATE TABLE IF NOT EXISTS webhook_events (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            event_source TEXT NOT NULL,
            payload TEXT NOT NULL
        )
    """)

    conn.commit()
    conn.close()

# Save a webhook event
def save_event(source, payload):
    conn = sqlite3.connect("webhooks.db")
    cursor = conn.cursor()

    cursor.execute("INSERT INTO webhook_events (event_source, payload)
VALUES (?, ?)",
                  (source, payload))

    conn.commit()
    conn.close()

```

Explanation

- `init_db()` → ensures the database and `webhook_events` table exist.
- `save_event(source, payload)` → inserts the event into the table.

Step 3: Create the Flask App

Create `app.py`:

```

from flask import Flask, request, jsonify
import json
from database import init_db, save_event

app = Flask(__name__)

# Initialize DB on server start
init_db()

# Stripe webhook endpoint
@app.route("/webhook/stripe", methods=["POST"])
def stripe_webhook():
    data = request.get_json()
    save_event("stripe", json.dumps(data))
    return jsonify({"status": "received"}), 200

# GitHub webhook endpoint
@app.route("/webhook/github", methods=["POST"])
def github_webhook():
    data = request.get_json()
    save_event("github", json.dumps(data))
    return jsonify({"status": "received"}), 200

if __name__ == "__main__":
    app.run(port=5000, debug=True)

```

Explanation

- /webhook/stripe → receives Stripe events.
- /webhook/github → receives GitHub events.
- Both store data into SQLite with source + JSON payload.
- Responds with {"status": "received"} to confirm success.

Step 4: Run the Flask Server

```
python app.py
```

Server starts on: <http://127.0.0.1:5000>

Step 5: Simulate Webhook Events

Send a **Stripe-like webhook**:

```
curl -X POST http://127.0.0.1:5000/webhook/stripe \  
  -H "Content-Type: application/json" \  
  -d '{"event": "payment_success", "amount": 250}'
```

Response:

```
{"status": "received"}
```

Send a **GitHub-like webhook**:

```
curl -X POST http://127.0.0.1:5000/webhook/github \  
  -H "Content-Type: application/json" \  
  -d '{"event": "push", "repo": "demo-repo}"'
```

Response:

```
{"status": "received"}
```

Step 6: Verify Stored Events

Open SQLite DB: `sqlite3 webhooks.db`

Run query: `SELECT * FROM webhook_events;`

Sample output:

```
1 | stripe | {"event": "payment_success", "amount": 250}
2 | github | {"event": "push", "repo": "demo-repo"}
```

Both webhook events are saved in the database.

Verification Checklist

- Database created with `webhook_events` table.
- Flask app exposes `/webhook/stripe` and `/webhook/github`.
- Webhook POST requests return `{"status": "received"}`.
- Data stored successfully in SQLite DB.

Summary

In this lab, you:

- Built a Flask server with **dedicated webhook endpoints**.
- Stored webhook payloads into an **SQLite database**.
- Simulated webhooks using `curl`.

With this setup, you can now:

- Extend to other webhook providers (Slack, Twilio, PayPal).
- Add **authentication checks** (secret tokens, HMAC signatures).
- Forward events to **queues (Kafka, RabbitMQ)** for async processing.